

Homework 3 Report

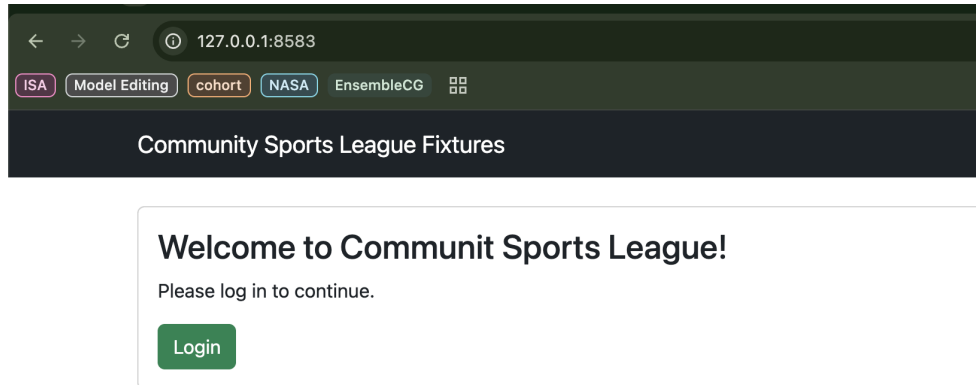
Section 0 - Personal Configuration and Domain

Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Local model	qwen3:8b through Ollama
Python version	3.11.9
Tagged commit hash	

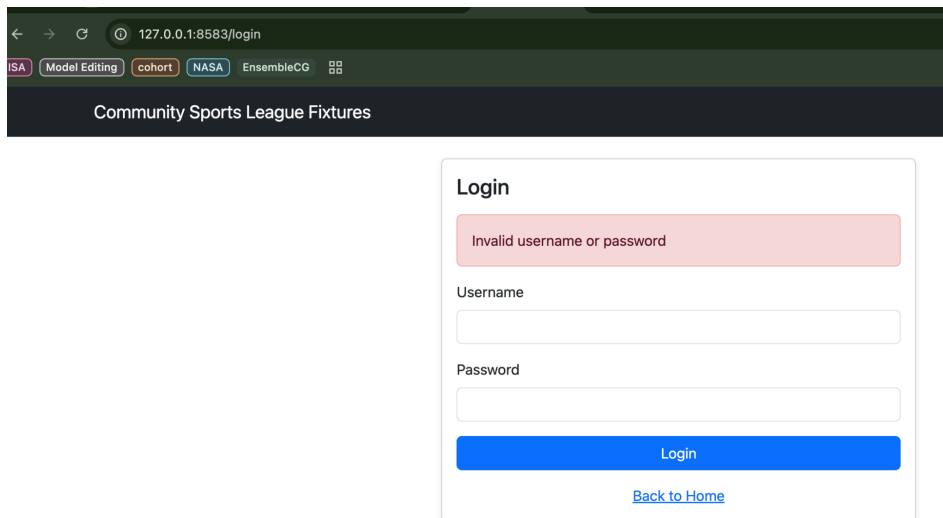
Part 1

1. Routes - Handled Separately in a router

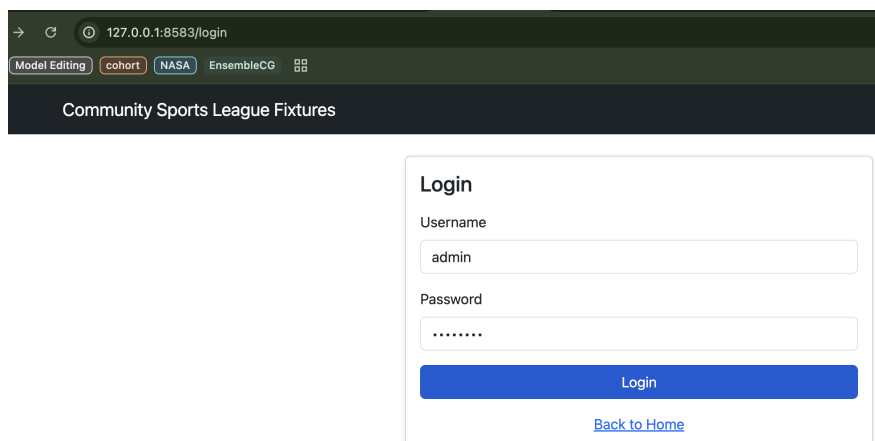
a. Home



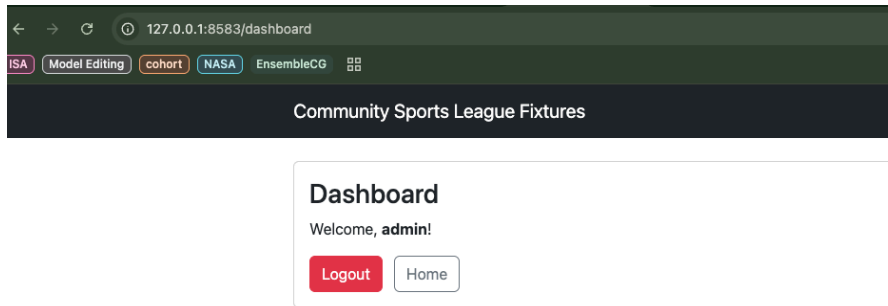
b. Login



Temporary login check with valid credentials produced no bootstrap error:

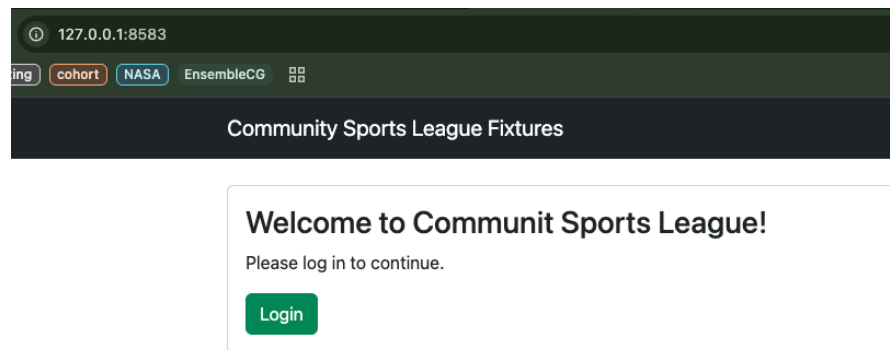


c. Dashboard



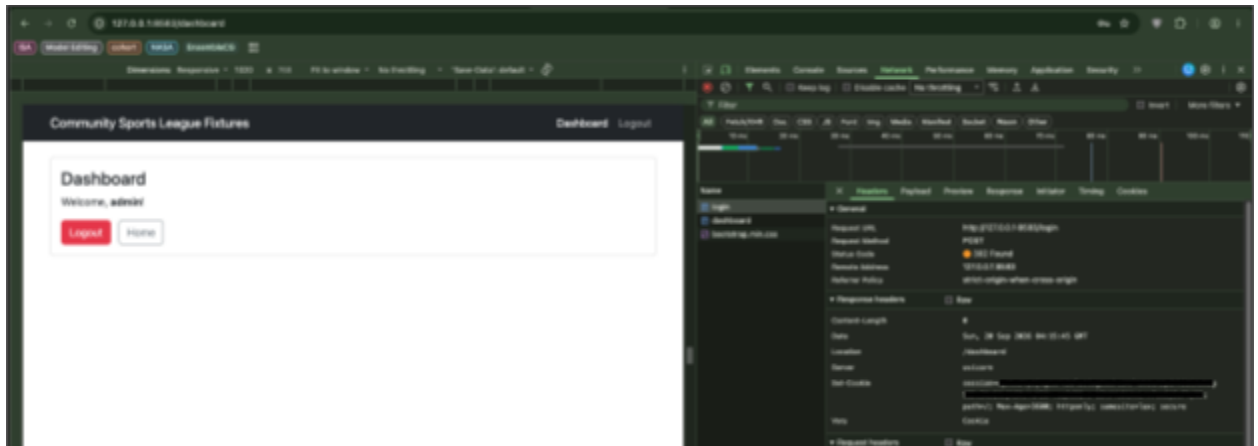
d. Logout

After clicking logout on the previous page:

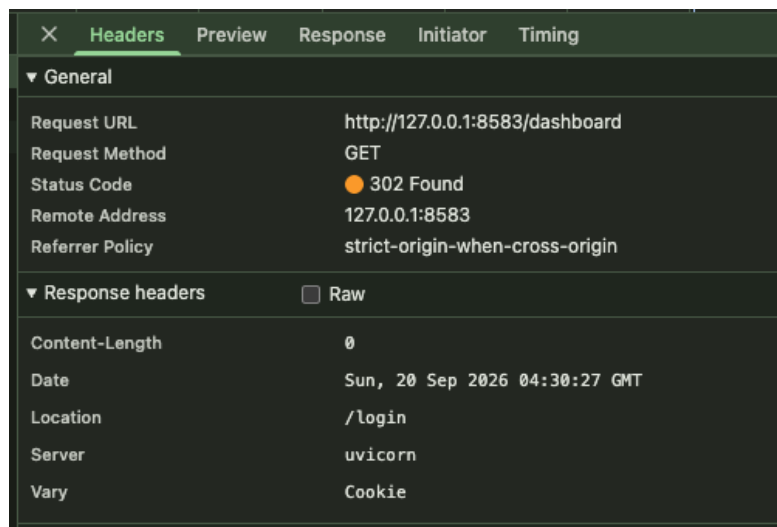
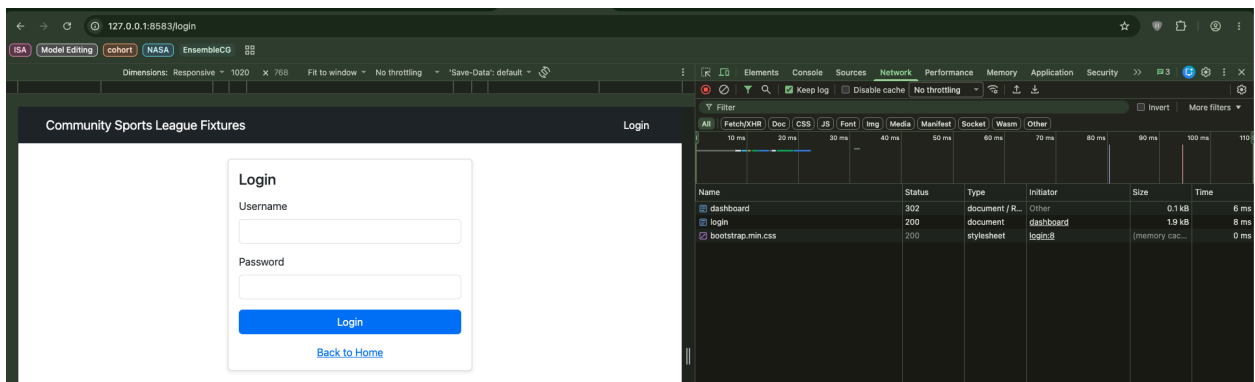


2. Session Management

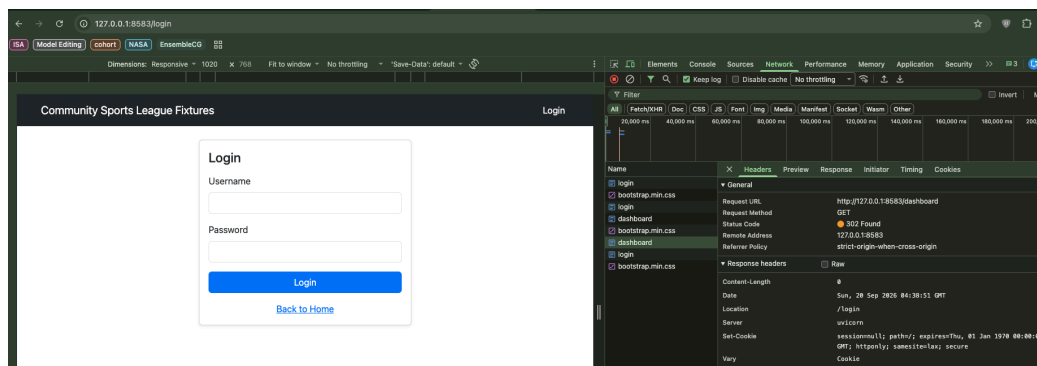
Successful login response showing the Set-Cookie header with the session value redacted and the required Secure, HttpOnly, SameSite=Lax, and Max-Age attributes.



After logout, accessing the protected /dashboard route returned a 302 redirect, followed by a successful request to /login, confirming that unauthenticated users cannot access the dashboard.



After the session exceeded the 60-second idle timeout (just for a test), a request to the protected /dashboard route returned 302 Found with Location: /login. The response also cleared the session cookie by setting session=null with an expired date.



auth.py

```
from urllib3.util import request
from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND
from pathlib import Path
import time

# IDLE_TIMEOUT_SECONDS = 60 # temporary to test
IDLE_TIMEOUT_SECONDS = 3600

# Create a router object
# This behaves like a mini FastAPI app
router = APIRouter()

# Configure Jinja2 templates directory
APP_DIR = Path(__file__).resolve().parent.parent
templates = Jinja2Templates(
    directory=str(APP_DIR / "templates")
)

# Hardcoded credentials for demo purposes only
# In real applications, credentials come from a database
VALID_USERNAME = "admin"
VALID_PASSWORD = "password"

@router.get("/")
def home(request: Request):
    """
    Home page route.

    - Checks if a user is logged in using the session
    - Passes user info to the template
    """
    user = request.session.get("user")

    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={
```

```

        "user": user
    }
)

@router.get("/login")
def login_page(request: Request):
    """
    Displays the login form.

    If the user is already logged in,
    the template can choose what to display.
    """
    user = request.session.get("user")
    error = request.session.pop("error", None)

    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "request": request,
            "user": user,
            "error": error
        }
    )

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        # Store logged-in user in session
        request.session["user"] = username
        request.session["last_activity"] = time.time()
        # Redirect user to dashboard

```

```

        return HttpResponseRedirect(
            url="/dashboard",
            status_code=HTTP_302_FOUND
        )

# If credentials are invalid:
# Redirect back to login page
request.session["error"] = "Invalid username or password"
# NOTE:
# No error message is shown intentionally.
# Students are expected to add Bootstrap alerts.
return HttpResponseRedirect(
    url="/login",
    status_code=HTTP_302_FOUND
)

@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected route.

    - Only accessible if user is logged in
    - Redirects to login page if session is missing
    """
    # user = request.session.get("user")
    user = get_active_user(request) # call the middleware function

    # If user is not logged in, block access
    if not user:
        return HttpResponseRedirect(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # If user is logged in, render dashboard
    return templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={
            "request": request,
            "user": user
        }
    )

```

```

    }

)

@router.get("/logout")
def logout(request: Request):
    """
    Logs the user out.

    - Clears all session data
    - Redirects back to home page
    """
    request.session.clear()

    return RedirectResponse(
        url="/",
        status_code=HTTP_302_FOUND
    )

# Middleware helpers
def get_active_user(request: Request):
    user = request.session.get("user") # checks if the user is logged in
    last_activity = request.session.get("last_activity") # checks when the user
was last active

    if not user or not last_activity:
        return None

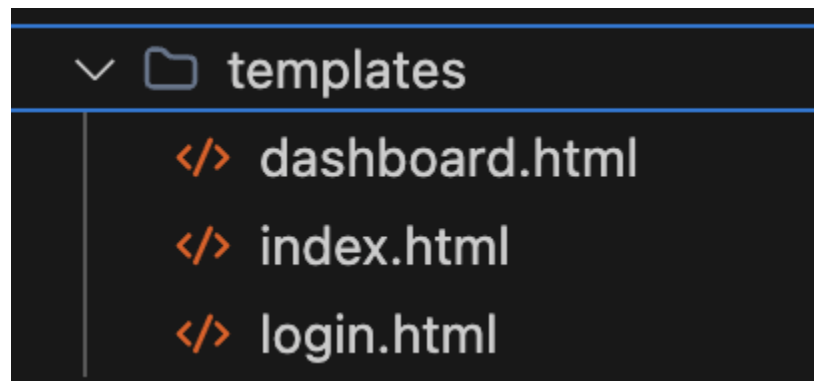
    if time.time() - last_activity > IDLE_TIMEOUT_SECONDS:
        request.session.clear() # if user is inactive for too long, clear the
session
        return None

    # Refresh activity time while the session is active
    request.session["last_activity"] = time.time() # if user is active, refresh
the last activity time

    return user

```

Templates directory:



Part 2

Three LlamaIndex Chunking Techniques

1. Retrieval Quality

Technique	Chunks	Average Chunk Length	Top 1 cosine	Mean @k cosine	Recall @k	Mean retrieval latency (ms)
Token	120	461.90	0.633	0.584	1.0	17.72
Semantic	21	2213.95	0.626	0.557	1.0	9.37
Sentence Window	254	183.04	0.596	0.563	1.0	12.96

Looking at the JSON files generated for the retrievals per question and technique, one example where the response was ranked number 1 but was incorrect is question 2 with sentence-window chunking. The question asks for application deadlines for different seasons, but the retrieved chunk does not contain the answer. The embedding might have been considered similar because it also contains permit information, although it refers to Sunday games, and mentions a year, which is related to the seasonal question.

Upon observation, one limitation of the current pipeline is that there are artifacts related to links and navigation text in the HTML, as well as some conversion errors in the PDF text. These issues may have affected the embeddings and retrieval results for some questions.

2. Observations

Looking at the JSON files generated for the retrievals per question and technique, the best technique was not the same for every question.

- For question 1, token chunking had the highest mean@5 cosine similarity, but the chunk containing the seasons, nights, and divisions was ranked 3.
- For question 2, token chunking performed better because the chunk containing the seasonal deadlines was ranked 2, while the other techniques retrieved different permit information in their top results.
- For question 3, sentence-window chunking performed better because the rank 1 chunk directly listed the required roster information.
- For question 4, sentence-window chunking retrieved the correct Fall 2026 tennis PDF at rank 1.
- For question 5, token chunking retrieved the relevant league and field-status information within the top five results, but not all of the answer was present in the rank 1 result.

Upon observation, it was noted that token chunking worked better when the answer included multiple separate facts, while sentence-window chunking worked better when the answer was present in one complete sentence or when nearby context was needed. Semantic chunking was generally faster, but it did not always retrieve the most relevant part of the answer.

3. Conclusion

Based on the summary metrics, we can conclude that token chunking is relatively better as compared to the other 2 techniques. This is because it had the highest top 1 cosine (0.633) and mean k cosine (0.584) while also successfully identifying the source like the other two. I think this is because the corpus generally consisted of factual information and the questions were also targeted to retrieve the same. Token chunking keeps nearby details together and also preserves exact words.

AI Usage:

An AI assistant was used to correct the grammar and sentence structure of the text in this report. The main analysis and observations were my own.